



UNIVERSITY OF
CAMBRIDGE

Department of Computer
Science and Technology

Research project report title page

Candidate 1234N

“Scheduling of Distributed LLM Serving on Heterogeneous GPUs”

Submitted in partial fulfillment of the requirements for the
Master of Philosophy in Advanced Computer Science

Total page count: 46

Main chapters (excluding front-matter, references and appendix): 36 pages (pp 9–44)

Main chapters word count: 6563

Methodology used to generate that word count:

[For example:

```
$ make wordcount
gs -q -dSAFER -sDEVICE=txtwrite -o - \
    -dFirstPage=9 -dLastPage=43 report-submission.pdf | \
    egrep '[A-Za-z]{3}' | wc -w
6563
```

]

Abstract

Write a summary of the whole thing. Make sure it fits on one page.

Contents

1	Introduction	9
2	Background	10
2.1	Transformers	10
2.1.1	Computation Steps	10
2.1.2	Prefill and Decode	11
2.1.3	Key Value caching	11
2.1.4	Mixture of Experts	11
2.1.5	DeepSeek Architecture	12
2.2	Parallelism	12
2.3	Scheduler	12
2.3.1	Heuristic-Based Methods	13
2.3.2	Metaheuristic Algorithms	13
2.3.3	Search-Based	13
2.3.4	Constraint Programming	13
3	Related work	15
3.1	Case Studies	15
3.1.1	Thunderserve	15
3.1.2	HexGen	15
3.1.3	DistServe	15
3.1.4	Splitwise	15
3.1.5	Summary	15
3.2	Future	15
4	Design and implementation	16
4.1	Simulator	17
4.1.1	Shallowsim Implementation	17
4.1.2	Modifactions	18
4.1.3	Usage	18
4.2	Workload Modeling	19
4.2.1	Input Sequence Length	19

4.2.2	Decode Length	19
4.3	Objectives	21
4.3.1	Throughput Modelling	21
4.3.2	Throughput Allocation	22
4.4	Constraint Optimisation	23
4.4.1	Problem Formulation	23
4.4.2	Problem Analysis	24
4.5	Scheduling Assumptions	25
4.5.1	Prefill Parallelism	25
4.5.2	Decode Parallelism	26
4.5.3	Solver Resolution	28
4.6	System Architecture	31
4.7	Inner Loop	32
4.7.1	Linear Representation	32
4.8	Outer Solver	33
4.8.1	Direct Island Generation	33
4.8.2	Divider Island Generation	35
5	Evaluation	38
5.1	Inner Loop	39
5.2	Outer Loop	41
5.2.1	Traces	42
6	Summary and conclusions	44
A	Technical details, proofs, etc.	46
A.1	Lorem ipsum	46

List of Figures

4.1	Sample probability distribution of input sequence lengths	19
4.2	Sample probability distribution of decode lengths	20
4.3	Selection of GPUs benchmarked over the prefill stage using the <i>Shallowsim</i> simulator	25
4.4	Selection of GPUs benchmarked over the decode stage using the <i>Shallowsim</i> simulator	28
4.5	System Architecture	31
5.1	Sample Benchmark	39
5.2	Sample Assignment	39
5.3	Sample Throughput	40
5.4	Comparison of BO methods	41
5.5	Context Tokens	42
5.6	Decode Tokens	42
5.7	Comparison of Conversation and Code	42
5.8	Context Tokens	43
5.9	Decode Tokens	43
5.10	Comparison of Conversation and Code	43

List of Tables

Chapter 1

Introduction

LLMs used everywhere

Inference for LLMs has energy and power requirements

Optimising this is important for the future

Heterogenous is better

Challenge is scheduling, that is what we look into

Specifically MoE

Chapter 2

Background

2.1 Transformers

Transformer models are a type of deep learning architecture specifically designed to handle sequential data such as natural language. Unlike recurrent neural networks (RNNs) which process sequences one step at a time, transformers can analyse entire sequences of data in parallel. This is achieved through self-attention, where each token in a sequence can weigh and incorporate information from all other tokens. Such attention is calculated by adding positional encodings to input embeddings which transforms each token into query, key and value vectors. Some model architectures also use multi-head attention, where multiple query, key and value vectors are calculated.

The first Transformer models were originally designed for encoder-decoder tasks such as translation, where an input sequence is encoded and decoded to an output sequence. An encoder Processes input sequence Contextualized hidden representation No access to future tokens Decoder Generates output tokens one-by-one Masked self-attention to prevent seeing future tokens Encoder-only models are typically used for classification and sentence embeddings. Decoder-only Used for autoregressive generation (language modelling, chat-bots, coding assistants Transformers form the basis of modern natural language processing due to their scalability, parallelisation benefits, and excellent performance on long-range dependencies. What is an LLM?

2.1.1 Computation Steps

There are several compute steps involved in a Transformer model's forward pass, during inference tasks. The process begins with Input Embedding and Positional Encoding where each token in the input sequence is converted into a vector, using a learned embedding matrix. Next, the model computes the Query (Q), Key (K) and Value (V) vectors for each token, this is calculated by projecting the input embeddings through learned linear transformations.

Models can also use Multi-Head Attention Instead of computing a single attention operation, the model splits the Q, K, and V matrices into multiple “heads”, concatenates the outputs The Add and Normalisation steps follow attention - The original input is added to the output and normalised Feed-Forward Network Stacked Layers Output

2.1.2 Prefill and Decode

We can separate the processing phases of LLM model inference steps into two distinct categories, with clear differences in hardware processing requirements.

The prefill stage processes the input tokens to compute the transformer model’s intermediate states. This includes calculating the positional embeddings and populating the KV cache, which are subsequently used to generate the first new token.

This specific phase is easily parallelized because at every stage of computation (per token) the full input sequence is known. Hence, this is viewed as a matrix-matrix operation which can easily saturate a GPU’s available compute. *In reference to a transformer’s computation steps, all stages are performed for every token in the sequence but can be heavily parallelised.*

The decode phase generates each output token one at a time, this is because each token computation is dependent on the previous token. Hence, each token must be calculated sequentially. The speed at which the decode phase can execute is heavily dependent on the speed at which weights, keys, values and activations can be loaded into the correct areas of GPU memory. Hence this phase usually saturates the memory bandwidth of a GPU before saturating the compute performance. *In reference to a transformer’s computation steps, all stages are performed, but for only the new token generated - this is due key-value caching.*

2.1.3 Key Value caching

Key value caching is an optimisation used during inference steps for transformer-based decoder-only models such as GPT. This technique avoids recomputing the tensors for each input token for every decode token generated. Instead weights and key values are cached in GPU memory, to be used in the next iteration. At each decoding step the Q value of the new token is computed, reusing cached K and V values from earlier tokens in the sequence. This technique comes at the cost of using more GPU memory, reducing the maximum model size that theoretically could be loaded.

2.1.4 Mixture of Experts

A neural network architecture specifically design to scale up models Instead of one big model Pool of expert networks For each input token, usually only a subset of these experts are activated Router chooses which experts to use based on the input Designed to be

compute efficient Only the activate experts consume compute Allows for scaling up a model's capacity (without linear increase in inference cost) Good Big model cpacpaity Better sceling lwas Felxiblke Bad Routing instability Load balancing Complexity

2.1.5 DeepSeek Architecture

Some information about the DeepSeek architecture. Focus on MoE, A2A etc.

2.2 Parallelism

Data parallelism refers to replicating a model across multiple processing units (GPUs / CPUs / TPUs), with each replica receiving a different slice of the input data. Hence, this technique is generally used to increase throughput of an overall system, especially in batched inference scenarios

Model parallelism involves partitioning the model itself across several processing units, where each device is responsible for a portion of the model's layers or operations. This is most helpful when a model cannot fit on a single device, due to memory demand greater than what is available.

Enables inference on devices that individually do not support Every forward pass requires synchronisation, introducing a communication overhead that may negatively impact latency Example

Tensor parallelism is a specialised form of model parallelism, generally used with large deep learning models. Individual tensors are split across multiple devices (rather than whole layers Operator-Level parallelism Stuff

Expert Parallelism

Mixture of experts Expert parallelism occurs when different experts are placed on different devices and are activated in parallel for

Batching

What are the different types of batching, how does this affect the overall performance on the system? Focus on latency, 99th percentile

2.3 Scheduler

Context on why talking about schedulers...

Optimisation techniques...

2.3.1 Heuristic-Based Methods

What is this kind of technique?

First-Come, First-Served (FCFS) is...

Round Robin (RR) is...

Min-Min / Max-Min is...

Priority-Based is...

Load Balancing (Best Fit, Worst Fit) is...

2.3.2 Metaheuristic Algorithms

An intro...

Genetic Algorithms (GA) is...

Particle Swarm Optimization (PSO) is...

Ant Colony Optimisation (ACO) is...

Simulated Annealing (SA) is...

2.3.3 Search-Based

Search-based Optimisation refers to methods that systematically or semi-randomly explore a given search space to find an optimal solution. Such techniques are often used to tune a scheduler's parameters, but can also

Bayesian Optimisation (BO) Aquastition function Approximate function

Random Search Sample search space uniformly at random + evaluate Context - randomly generate candidate schedules, and pick the best one found Search space is significant

Grid Search Brute force Discrete set of possible values for each parameter, then evaluates every combination Suffers from curse of dimensionality

2.3.4 Constraint Programming

Both Constraint Programming and Mathematical Optimisation approaches formulate scheduling activities as an optimisation problem, with formal mathematical constraints and objectives.

Linear Programming (LP) techniques can solve scheduler problems where the constraints and objectives can be modelled as only linear. All decision variables are continuous and can be used in scheduling algorithms when fractional assignments of work are valid.

Mixed Integer Linear Programming (MILP) solvers can consider integer variables such as binary assignment values, extending LP. In the context of scheduling, this enables decisions to be modelled using yes/no decisions and combinatorial choices.

In the case where constraints cannot be as linear, **Non-Linear Programming (NLP)** can model problems where either the objective and/or constraints are non-linear functions. Given problems are non-linear, it is harder to guarantee a globally optimal solution is generated.

Mixed Integer Programming (MIP) is...

Chapter 3

Related work

3.1 Case Studies

3.1.1 Thunderserve

Thunderserve [1]

3.1.2 HexGen

HexGen [2]

3.1.3 DistServe

DistServe [3]

3.1.4 Splitwise

Stuff [4]

3.1.5 Summary

Stuff (General trends)

3.2 Future

Stuff (Possible Improvements)

Chapter 4

Design and implementation

Explain GPU islands... Scheduler direct traffic... Configure GPUs...

4.1 Simulator

In order to be able to develop an advanced heterogenous LLM scheduler, we need access to many high-performance GPUs to test our schedule configurations. Unfortunately, GPU-hours are prohibitively expensive and using such hours for “dummy” workloads to test a scheduler would be considered a waste of resources. Instead, we utilise the latest simulator technology to estimate the performance of LLM inference given a set of input parameters. Several simulators exist that attempt to model the performance of an LLM request by using estimation techniques.

Vidur [5] (essentially uses linear regression)

Shallowsim [6] is a specialist simulator designed to estimate the compute times for different LLM request types.

LLMCompass [7] (Need to look up)

We Select Shallowsim for its transparency for calculations (simplicity). And support for Mixture of Expert models

4.1.1 Shallowsim Implementation

Prefill

Dense-MLA (dense Multi-head/Multi-query Attention compute)

TP-MLA (Dense-MLA plus the Tensor-Parallel all-reduce)

Dense-MLP (feed-forward network compute)

Shared-Expert (MoE expert run that every token visits) Routed-Expert (MoE expert run that only selected tokens visit)

Every token first passes through the Shared Expert, but a subset of tokens are also routed to extra specialists. This is modelled with the Routed Expert function.

Dispatch (All-to-All that ships tokens out to remote experts) Combine (All-to-All that brings tokens back)

Sending activations to the respective routed expert nodes is modelled using the Dispatch step, the Combine step then models how processed activations are collated back together. These two functions model the network overhead of LLM queries.

Compute Time - Dense MLA, Dense MLP (Dense Layers) - TP-MLA, Shared Expert, Routed Expert (Sparse Layers) Communication Timemake - Dispatch, Combine (Sparse Layers) Overlap Penalties - If Combine is longer than TP-MLA + Shared Expert - If Dispatch is longer than Routed Expert Sum - Compute + Communication + Any Overlap = Prefill Time

Decode

Load-KV Dense-MLA (as prefill) Sparse-MLA Dense-MLP (as prefill) Shared-Expert (as prefill) Routed-Expert (as prefill) Dispatch (as prefill) Combine (as prefill)

Compute Time - Dense MLA, Dense MLP (Dense Layers) - Sparse MLA, Shared Expert, Routed Expert (Sparse Layers) Communication Time - Dispatch, Combine (Sparse Layers) Overlap Penalties - TODO Sum - TODO = Per token decode time

4.1.2 Modifications

What we have done to modify the shallowsim simulator.

As designed, shallowsim is build for profiling several different GPU configurations at once.

For our system, we are particularly interested in per-configuration results

4.1.3 Usage

Prefill Phase

Using the Shallowsim simulator we can measure the time t_{ms} (in milliseconds) to compute a single prefill request for a given set of parameters, and convert this to throughput R (in requests per second) via $R_{\text{prefill}} = \frac{1000}{t_{\text{ms}}}$.

Decode Phase

Shallowsim has support for calculating the time in milliseconds for a single token generation step. As we would typically expect several tokens to be generated, we cannot directly compute requests per second using Shallowsim, instead we must perform additional computations. The decode time of a single token is affected by the input/current sequence length, therefore we must measure the single token compute time for every token to be generated.

By measuring $t_{\text{ms},i}$ the compute time (in milliseconds) for the i th token, the total decode time for a request generating N tokens is $t_{\text{decode}} = \sum_{i=1}^N t_{\text{ms},i}$.

We can then convert this total time into throughput R_{decode} (in requests per second), using the same method as prefill. $R_{\text{decode}} = \frac{1000}{t_{\text{decode}}} = \frac{1000}{\sum_{i=1}^N t_{\text{ms},i}}$, where R_{decode} represents how many full decode requests per second the configuration can sustain.

4.2 Workload Modeling

We expect LLM inference services to encounter several different classes of workloads, depending on the user’s use-case. These workloads have different characteristics of input sequence length (request message plus any system prompts), and also different decode lengths (numbers of characteristics generated). For example, one would expect code prompts and responses to have different input sequence and decode lengths than to that of conversation prompts. Hence, we must design a scheduler that is able to optimise over a specific workload class.

4.2.1 Input Sequence Length

To achieve fine-grained control of request scheduling, dependant on input sequence, we model the input request length as a probability distribution. Hence, for a chosen workload type, there is a specific probability of request arriving with given sequence length.

Let $w \in W$ denote a workload type from the set of all workload types W . For each workload type w , we define a probability mass function $p(s \mid w)$, which represents the probability of a request having sequence length $s \in S$ under workload w . This distribution satisfies: $\sum_{s \in S} p(s \mid w) = 1$ for all $w \in W$.

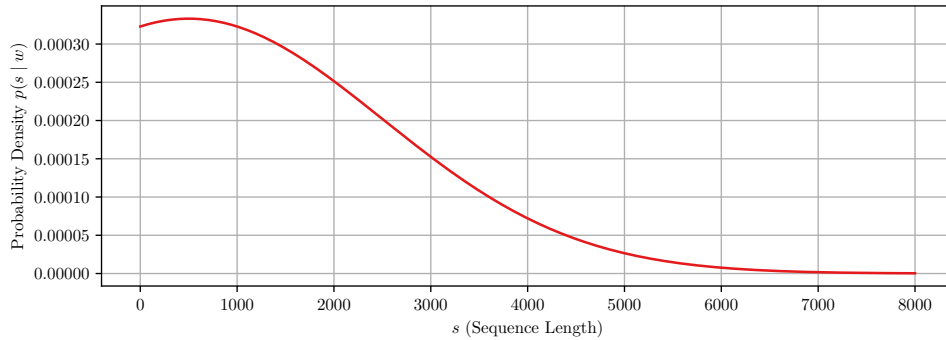


Figure 4.1: Sample probability distribution of input sequence length. Modelled using a pdf where $\mu = 500$, $\sigma = 2000$ and truncated/normalised to the range $0 \leq s \leq 8000$.

Instead of only considering the distribution of sequence lengths during assignment, to model system throughput, we use them to find the most optimal pairing of sequence length to GPU island. Therefore, specific GPU islands can be configured to most optimally perform prefill operations for a specific sequence length. When a request arrives, we can direct it to the GPU islands targeted to serve that request length, defined in the schedule.

4.2.2 Decode Length

While decode length can also be modelled using a probability density function, this information (specific distribution) is less useful from a scheduling perspective. In contrast to input sequence length, the decode length is not known the moment a request arrives.

Instead, the generation process proceeds token-by-token until an explicit or implicit stop condition is met (e.g. end-of-sequence token).

We can assume that a decode request must complete on the GPU island it started processing on, as switching island would incur additional communication overhead and significantly impact the request response time.

For some workloads there is a small correlation between input sequence length and decode length, but this relationship is often not significant. Therefore, at the point of request arrival we do not have enough information about the decode length to make informed per-decode length scheduling decisions. Hence, creating specific GPU configurations for short, medium, long, etc. decode lengths would likely be inefficient. As we cannot accurately predict decode length, there is a high probability that a decode request would have to move island.

Instead, we model our scheduler using an average decode length for a given workload type and design every GPU decode island to support the maximum decode length expected. Unlike the sequence length, the decode length is simply being used for system modelling activities to characterise performance, not direct traffic. However, we can still make scheduling decisions for the decode phase using the sequence length as a feature, given the maximum decode length affected by the input length. This is due to all the input and decode tokens KV values being stored in GPU memory.

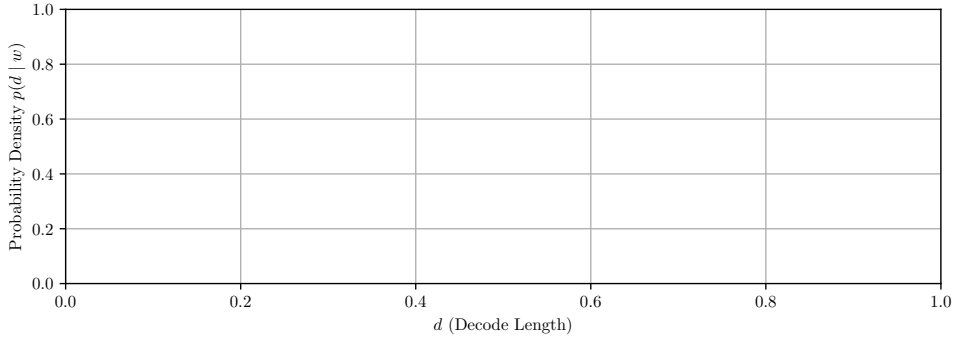


Figure 4.2: Sample probability distribution of decode length. Modelled using a pdf where $\mu = 100$, $\sigma = 200$ and truncated/normalised to the range $0 \leq s \leq 500$. Average decode length (100) and max decode length (500) are extracted for use in the scheduler.

4.3 Objectives

4.3.1 Throughput Modelling

Demonstrated in subsection 4.1.3, the Shallowsim simulator has support for estimating the throughput of a single request of a static input sequence and decode length. It is highly improbable that an entire workload consists of a single request type, instead we expect a distribution of sequence lengths, as shown in section 4.2. Hence, we need a method to accurately calculate the throughput of a system, with a variety of input sequence.

Using the probability distribution of the workload, we can compute the weighted average of throughput, using measured performance values for each sequence length. Let $R_{\text{prefill}}(s)$ denote the measured prefill throughput (in requests per second) for a request of sequence length s . The expected prefill throughput for workload w , denoted $\mathbb{E}[R_{\text{prefill}} | w]$ (expectation), is then given by:

$$\mathbb{E}[R_{\text{prefill}} | w] = \sum_{s \in S} R_{\text{prefill}}(s) \cdot p(s | w)$$

In addition to prefill, we can also compute the expected throughput for the decode phase. While prefill throughput depends solely on the input sequence length s , decode throughput typically depends on both the input length s and the output (decode) length d . For simplicity, we assume a fixed average decode length d as a parameter. Let $R_{\text{decode}}(s, d)$ denote the measured decode throughput (in requests per second) for an input sequence length s and average decode length d . Given a workload-specific sequence length distribution $p(s | w)$, the expected decode throughput for workload w is:

$$\mathbb{E}[R_{\text{decode}} | w, d] = \sum_{s \in S} R_{\text{decode}}(s, d) \cdot p(s | w)$$

To account for the complete request processing pipeline, we define the overall expected throughput as the minimum of the expected prefill and decode throughputs. Since requests cannot be served faster than the slowest stage, the effective throughput is bounded by the bottleneck. Let $\mathbb{E}[R_{\text{prefill}} | w]$ denote the expected prefill throughput for workload w , and $\mathbb{E}[R_{\text{decode}} | w, d]$ the expected decode throughput for workload w and average decode length d . The overall expected throughput is given by:

$$\mathbb{E}[R_{\text{overall}} | w, d] = \min(\mathbb{E}[R_{\text{prefill}} | w], \mathbb{E}[R_{\text{decode}} | w, d])$$

This value represents the effective throughput achievable by the system under the given workload and output length assumptions.

4.3.2 Throughput Allocation

Previous calculations assumed that a singular GPU island is contributing to the overall throughput of the system. In reality, we would like to be able to construct several GPU islands and assign each island to a request type it is best suited for. Hence, we allocate a percentage of GPU resources for a given island to a given sequence length it specialises in. Let I be the set of all GPU islands in the system, which we partition into two separate subsets I_{prefill} and I_{decode} . Let $x_i(s)$ denote the fraction of overall GPU capacity on island i devoted to handling requests of length s . We normalise these allocations so that

$$\sum_{s \in S} x_i(s) = 1 \quad \text{for every } i \in I.$$

By including $x_i(s)$ into our previous per-length throughput models, the expected prefill throughput under workload w becomes

$$\mathbb{E}[R_{\text{prefill}} \mid w] = \sum_{i \in I_{\text{prefill}}} \sum_{s \in S} x_i(s) R_{\text{prefill}}^{(i)}(s) p(s \mid w),$$

while the expected decode throughput for average decode length d is

$$\mathbb{E}[R_{\text{decode}} \mid w, d] = \sum_{i \in I_{\text{decode}}} \sum_{s \in S} x_i(s) R_{\text{decode}}^{(i)}(s, d) p(s \mid w).$$

4.4 Constraint Optimisation

We can combine the previous throughput calculations to form a simple constraint optimisation problem with an objective to maximise the total expected throughput R and minimise the deviation Δ from the target workload distribution $p(s | w)$.

4.4.1 Problem Formulation

The following statement describes a sample non-linear optimisation problem that can be solved for either prefill or decode throughput.

Input Data

- I (set of GPU islands),
- S (set of sequence lengths),
- $b_i(s)$ (benchmark throughput of island i on length s),
- $p(s | w)$ (target probability of length s under workload w), $\sum_{s \in S} p(s | w) = 1$,
- λ (trade-off parameter, $\lambda > 0$, balancing throughput vs. deviation).

Decision Variables

$$x_i(s) \geq 0 \quad (\text{fraction of island } i\text{'s GPU capacity devoted to length } s).$$

Constraints

$$\sum_{s \in S} x_i(s) = 1 \quad \forall i \in I.$$

Intermediate Quantities

1. Performance per cell:

$$r_i(s) = b_i(s) x_i(s).$$

2. Total performance:

$$R = \sum_{i \in I} \sum_{s \in S} r_i(s) = \sum_{i \in I} \sum_{s \in S} b_i(s) x_i(s).$$

3. Performance by sequence length:

$$R_s = \sum_{i \in I} r_i(s) = \sum_{i \in I} b_i(s) x_i(s), \quad \forall s \in S.$$

4. Achieved share:

$$\sigma_s = \frac{R_s}{R}.$$

5. Deviation from target:

$$\delta_s = |\sigma_s - p(s \mid w)|, \quad \Delta = \sum_{s \in S} \delta_s.$$

Objective

Maximise

$$R - \lambda \Delta,$$

where

$$R = \sum_{i \in I} \sum_{s \in S} b_i(s) x_i(s), \quad \Delta = \sum_{s \in S} |\sigma_s - p(s \mid w)|, \quad \sigma_s = \frac{\sum_{i \in I} b_i(s) x_i(s)}{R}.$$

4.4.2 Problem Analysis

The constraint optimisation problem described is non-linear, this is due to the chained multiplications and divisions for the *performance per cell* and *achieved share* calculations. As a result, we require a non-linear programming (NLP) solver to obtain a solution for the proposed problem.

While the problem can be solved using such solvers, the process tends to be computationally intensive and slow. This is primarily due to the non-convex and non-linear nature of the formulation, which makes it difficult to guarantee convergence to the global optimum. Ideally, we would like to linearise this problem to improve computational efficiency and enhance the reliability of finding the best possible solution.

Furthermore, we statically define I to a set of curated islands, and the optimiser simply fits the workload to these islands. In order to generate the most optimal solution, we need to choose the best set of island sizes for each GPU type. However, adding this decision process to the constraint optimisation problem increases its complexity, as it effectively turns a bounded bin packing problem into an unbounded one. The optimiser must now determine not only the workload allocation but also the optimal sizes and number of islands. Hence, we need to formulate an architectural approach that incorporates this logic, which we detail in section 4.6.

4.5 Scheduling Assumptions

Before finalising a scheduler architecture, we can make several assertions and assumptions in an attempt to reduce the search space of the scheduler generation process. These effectively apply heuristics to our problem, that are crafted to simplify the computation requirements while retaining solution quality.

4.5.1 Prefill Parallelism

We can assume for a given model and GPU type, there exists an optimal parallelism configuration that is consistent across input sequence lengths. Demonstrated in Figure 4.3 we can see that for all GPUs, $tp = 64$ and $dp = 1$ is the fastest parallelism configuration for all sequence lengths above 128 tokens. Given we expect the majority of input requests to be over 128 tokens, we can reason that there is an *optimal* GPU prefill configuration for each GPU type.

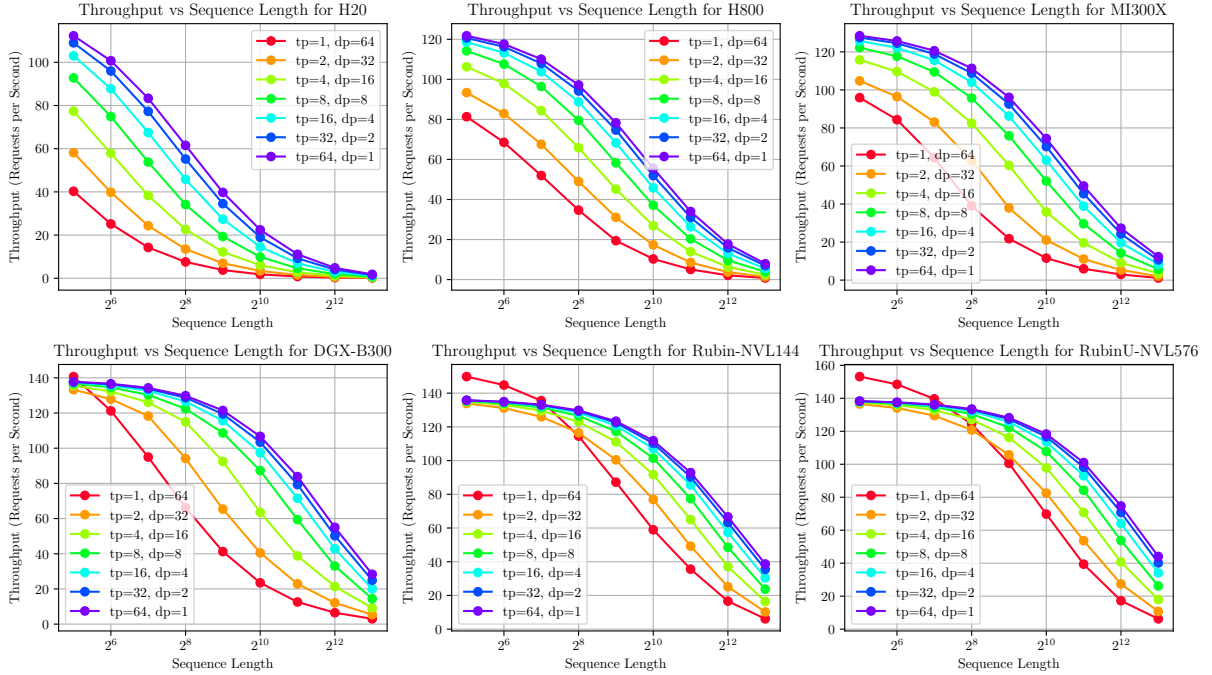


Figure 4.3: Selection of GPUs benchmarked over the prefill stage, with a variety of parallelism configurations using the *Shallowsim* simulator. All configurations use a static number of GPUs (64).

By pre-computing this optimal configuration, we significantly reduce the number of parameters considered by the scheduler. This is because an island cannot be formed with several different parallelism configurations simultaneously. And hence, the scheduler does not have to contain a variable for (and consider) each sequence-length-parallelism combination.

To identify a best GPU prefill configuration across sequence lengths we use the following calculations, let G be the total number of GPUs and consider the set of all parallelism

configurations $C = \{c = (tp, dp) \mid tp \cdot dp = G\}$. For each configuration $c \in C$ and sequence length $s \in S$, we measure the prefill throughput $R_{\text{prefill}}(s, c)$ and define the optimal throughput at length s by

$$R_{\text{prefill}}^*(s) = \max_{c \in C} R_{\text{prefill}}(s, c).$$

We then compute the relative deviation of configuration c at length s as

$$\delta(c, s) = \frac{R_{\text{prefill}}^*(s) - R_{\text{prefill}}(s, c)}{R_{\text{prefill}}^*(s)}.$$

Aggregating these deviations over all lengths gives

$$\begin{aligned} \bar{\delta}(c) &= \frac{1}{|S|} \sum_{s \in S} \delta(c, s), & (\text{uniform average over } S), \\ \tilde{\delta}(c) &= \sum_{s \in S} \delta(c, s) p(s \mid w), & (\text{weighted by workload distribution}). \end{aligned}$$

Finally, the best overall configuration is chosen as the one minimising the average deviation from the individual best configurations.

$$c_{\text{prefill}} = \arg \min_{c \in C} \tilde{\delta}(c).$$

4.5.2 Decode Parallelism

We can compute the same best configuration for the decode phase, with small modifications to the set of C configurations taking into account mixture-of-experts. Unlike prefill, dp (the data-parallelism factor) is not a native parameter of the Shallowsim simulator. Instead, we pre-compute how many GPUs to pass to the simulator and then scale the resultant throughput by dp . Let G be the total number of GPUs, tp tensor parallelism factor, and n the total number of routed experts in the model. We can form every pairing of valid factors using:

$$T = \{tp \in \{1, \dots, G\} \mid G \bmod tp = 0\}, \quad D = \{dp \in \{1, \dots, G\} \mid G \bmod dp = 0\},$$

$$C' = T \times D = \{(tp, dp) \mid tp \in T, dp \in D\}.$$

For each $(tp, dp) \in C'$, set

$$G_{\text{sim}} = \frac{G}{dp}, \quad ep = \left\lceil \frac{n}{G_{\text{sim}}} \right\rceil,$$

and define the simulator configuration

$$c = (G_{\text{sim}}, tp, ep) \in C.$$

Then, for each $c \in C$ and sequence length $s \in S$, the simulator returns

$$R_{\text{prefill}}(s; c),$$

and under dp -way data parallelism the effective total prefill throughput is

$$R_{\text{prefill}}(s; dp, c) = dp \times R_{\text{prefill}}(s; c) = dp \times R_{\text{prefill}}\left(s; \frac{G}{dp}, tp, ep\right).$$

Before calculating the best parallelism configuration out of the set, we also need to decide which batch size to use. Since we would like to optimise our configuration for overall throughput, we need to maximise the batch size for each input sequence length, such that we remain within the memory constraints of the island. Let $\max_bs(s; c)$ denote the maximum batch size returned by the simulator for sequence length s under configuration c . We then round down to the nearest multiple of 8 and clamp between 8 and 1024 to keep the batch size aligned to the GPU memory and compute capabilities. This ensures efficient hardware utilisation and avoids potential performance degradation caused by unaligned memory access or suboptimal parallelism.

$$bs(s; c) = \min\left(\max(\max_bs(s; c) - (\max_bs(s; c) \bmod 8), 8), 1024\right).$$

Similar to prefill, we seek a single best decode parallelism configuration across sequence lengths. For each simulator configuration $c \in C$ and sequence length s , we first compute the batch size $bs(s; c)$ as before, then measure the decode throughput $R_{\text{decode}}(s, d; c, bs(s; c))$. We define the optimal decode throughput at length s by

$$R_{\text{decode}}^*(s, d) = \max_{c \in C} R_{\text{decode}}(s, d; c, bs(s; c)),$$

and the relative deviation of configuration c at s by

$$\delta_{\text{decode}}(c, s) = \frac{R_{\text{decode}}^*(s, d) - R_{\text{decode}}(s, d; c, bs(s; c))}{R_{\text{decode}}^*(s, d)}.$$

Aggregating these deviations over all lengths gives

$$\begin{aligned} \bar{\delta}_{\text{decode}}(c) &= \frac{1}{|S|} \sum_{s \in S} \delta_{\text{decode}}(c, s), & (\text{uniform average over } S), \\ \tilde{\delta}_{\text{decode}}(c) &= \sum_{s \in S} \delta_{\text{decode}}(c, s) p(s | w), & (\text{weighted by workload distribution}). \end{aligned}$$

We can then select the decode configuration that minimises the average deviation across all expected sequence lengths,

$$c_{\text{decode}} = \arg \min_{c \in C} \tilde{\delta}_{\text{decode}}(c).$$

Shown in Figure 4.4, for each GPU there is generally a better parallelism configuration for most sequence lengths. Unlike prefill, this configuration is not consistent across GPU types, but rather significantly impacted by the GPU specifications. Configurations with zero throughput are due to the configuration exceeding the memory capabilities of the cluster.

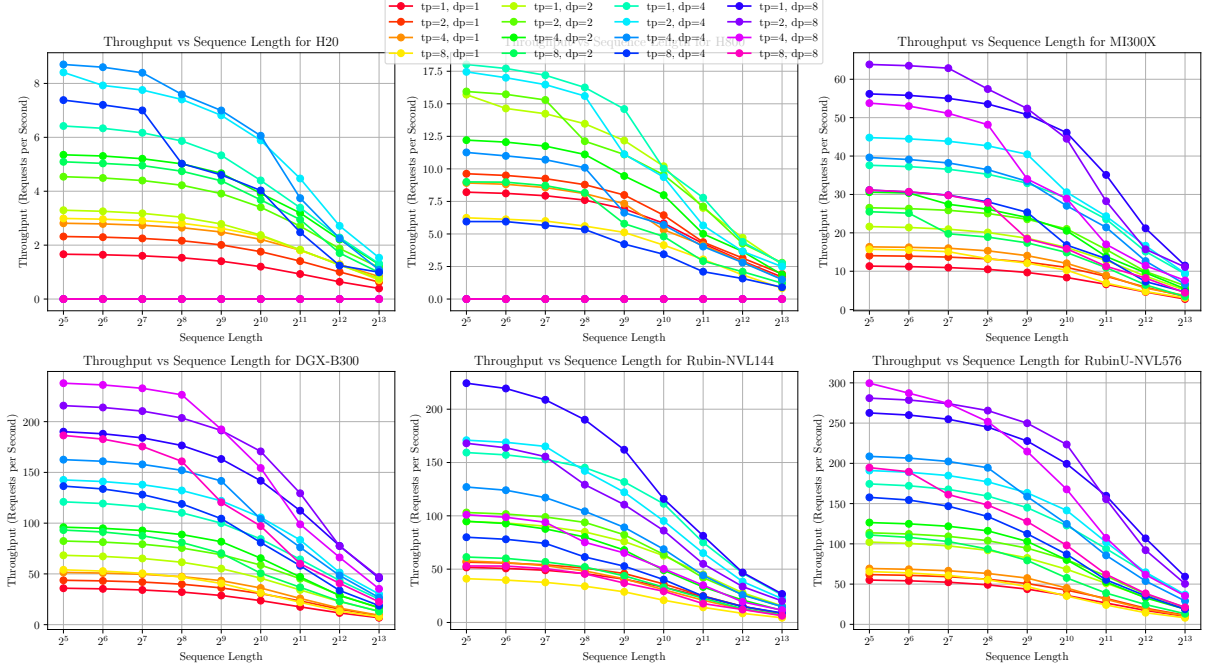


Figure 4.4: Selection of GPUs benchmarked over the decode stage, with a variety of parallelism configurations using the *Shallowsim* simulator. All configurations use a static number of GPUs (64).

4.5.3 Solver Resolution

We can use the simulator to benchmark the prefill and decode throughput of an island for a given input sequence length. While it is feasible to consider each sequence length ($s \in S$) individually (in our optimisation problem), we can use a binning approach to reduce the search space of the scheduler. The width of the ranges effectively control the resolution of the solution generated, which can be dynamically adjusted depending on input parameters.

We partition the sorted list of lengths S into equally-sized intervals of width W , called *ranges*. Rather than measure every s_i , we sample a single representative length from each range at its midpoint. With a positive integer width W , we compute the midpoint offset as $\text{mid} = \lfloor \frac{W}{2} \rfloor$. For each integer k such that $0 \leq k < \lfloor |S|/W \rfloor$, we let

$$\text{range_start}_k = k W, \quad \text{range_end}_k = k W + W, \quad \text{range_mid}_k = \text{range_start}_k + \text{mid},$$

and take range_mid_k as the representative length for the k -th range. We then also compute

the total probability of that range by summing

$$p_k = \sum_{i=\text{range_start}_k}^{\text{range_end}_k-1} p(s_i | w),$$

and record for range k the tuple $(\text{range_mid}_k, p_k, \text{range_start}_k, \text{range_end}_k)$. Therefore, the set of all ranges is

$$\mathcal{R} = \{ (\text{range_mid}_k, p_k, \text{range_start}_k, \text{range_end}_k) \mid 0 \leq k < \lfloor |S|/W \rfloor \}.$$

By sampling only at these midpoints and aggregating probabilities, we approximate the full set of sequence lengths in each range with a single point (range_mid_k) , thereby greatly reducing the number of simulator calls while preserving workload-weighted accuracy.

Prefill Throughput

With the formed ranges $\mathcal{R}$, each with their representative length s_r and total probability p_r , we can redefine the allocation variables $x_i(s)$ to act at the range level. Now island i assigns fraction $x_i(r)$ of its GPU capacity to range $r \in \mathcal{R}$, with

$$\sum_{r \in \mathcal{R}} x_i(r) = 1, \quad x_i(r) \geq 0 \quad \forall i \in I, r \in \mathcal{R}.$$

Within range r , every sequence length $s \in S_r$ is approximated by s_r , so that

$$\sum_{s \in S_r} R_{\text{prefill}}^{(i)}(s) p(s | w) \approx R_{\text{prefill}}^{(i)}(s_r) \sum_{s \in S_r} p(s | w) = R_{\text{prefill}}^{(i)}(s_r) p_r.$$

By including $x_i(r)$ into our per-range throughput models, the expected prefill throughput under workload w becomes

$$\mathbb{E}[R_{\text{prefill}} | w] = \sum_{i \in I_{\text{prefill}}} \sum_{r \in \mathcal{R}} x_i(r) R_{\text{prefill}}^{(i)}(s_r) p_r.$$

Decode Throughput

The same approach applies to the decode phase. For each range r with representative length s_r and probability p_r , we measure the decode throughput $R_{\text{decode}}^{(i)}(s_r, d)$ on island i for average decode length d . Island i allocates fraction $x_i(r)$ of its capacity to r , and within that range every sequence length is approximated by s_r . Thus,

$$\sum_{s \in S_r} R_{\text{decode}}^{(i)}(s, d) p(s | w) \approx R_{\text{decode}}^{(i)}(s_r, d) p_r.$$

Normalising $x_i(r)$ as before and summing over islands and ranges yields the expected decode throughput under workload w and decode length d :

$$\mathbb{E}[R_{\text{decode}} \mid w, d] = \sum_{i \in I_{\text{decode}}} \sum_{r \in \mathcal{R}} x_i(r) R_{\text{decode}}^{(i)}(s_r, d) p_r.$$

The overall expected throughput remains

$$\mathbb{E}[R_{\text{overall}} \mid w, d] = \min(\mathbb{E}[R_{\text{prefill}} \mid w], \mathbb{E}[R_{\text{decode}} \mid w, d]).$$

4.6 System Architecture

During section 4.4 we formulated a non-linear bounded bin-packing problem, by constraining the islands to a pre-defined type and size. In a true scheduling problem we expect to be allocated an inventory of different GPU types, let T be the set of GPU types. For each $t \in T$, let n_t be the number of GPUs of type t allocated in the inventory. The full GPU inventory is then the vector $(n_t)_{t \in T}$, with total number of GPUs in the system modelled as $N = \sum_{t \in T} n_t$.

Similar to ThunderServe [1], we can separate this scheduling problem as two separate, distinct stages. The first stage is the island layout generation, which we can treat as a search-like optimisation problem in which we explore combinations of GPU island sizes and types. The second stage is the bounded bin-packing step, a deterministic problem that assigns workload ranges to the pre-computed islands to maximise throughput for a given workload. By decoupling layout design from the packing phase, we can leverage specialised solvers for each subproblem and significantly reduce overall computational complexity.

We refer to the bounded bin packing problem as the *inner loop*, and the outer optimization process as the *outer loop*, as illustrated in Figure 4.5. The *outer loop* is responsible for generating promising island layouts, which are then passed to the *inner loop* for resource packing. The resulting throughput is returned to the *outer loop*, allowing the optimisation process to evaluate and refine its search for effective configurations. This iterative cycle can be repeated multiple times to generate an optimal solution.

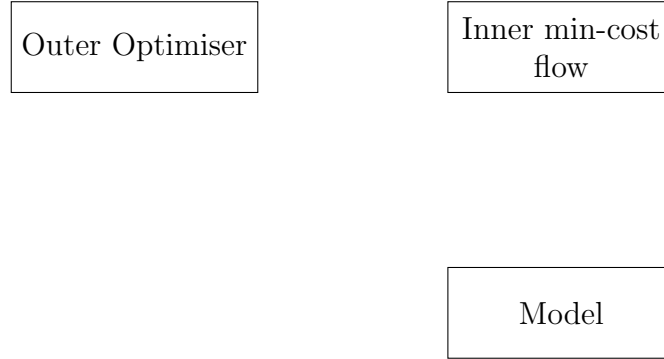


Figure 4.5: System Architecture

4.7 Inner Loop

The *inner loop* receives an island layout proposal from the *outer loop*, this layout is specified as a finite index set $\mathcal{I} = \{i = 1, \dots, |\mathcal{I}|\}$, where each island i is represented by a pair (t_i, s_i) . Here, $t_i \in T$ denotes the GPU type assigned to island i , and $s_i \in \mathbb{Z}_{\geq 0}$ indicates the number of GPUs of type t_i allocated to that island. The outer loop guarantees that these allocations respect the total-inventory constraints.

Given the predefined island layout $\{(t_j, s_j)\}_{j=1}^{|\mathcal{I}|}$, the inner loop's only task is to assign workloads of different prefill and decode lengths. No further modification of t_i or s_i takes place at this stage, as the inner loop focuses entirely on determining the optimal bin-packing configuration.

We can outline the steps of the *inner loop* as follow:

1. **Form Ranges** Take the specific workload distribution w , and separate into a series of ranges $\mathcal{R}$ using a specified resolution (subsection 4.5.3)
2. **Prefill Configuration Generation & Benchmark** Identify the best prefill configuration (c_{prefill}) for each island i , and return the benchmark results describing the throughput for each range (subsection 4.5.1).
3. **Decode Config Generation & Benchmark** Identify the best decode configuration (c_{decode}) for each island i , and return the benchmark results describing the throughput for each range (subsection 4.5.2).
4. **Problem Specification** Formulate the exact optimisation problem based on the provided prefill and decode benchmarks (??).
5. **Solve** Invoke the optimiser on the specified problem, to generate the best solution given the input parameters.
6. **Results** Extract the final workload-assignment solution from the solver's output, including any intermediary variables for debugging.

4.7.1 Linear Representation

We can take the learning from the previous optimisation problem (section 4.4) to linearise our problem and combine prefill with the decode phases. We can also take the learning from the solution resolution (subsection 4.5.3), to reduce the number of variables and the problem complexity.

4.8 Outer Solver

We consider a resource allocation problem where the goal is to optimise a black-box objective function $f : \mathcal{X} \rightarrow \mathbb{R}$ using Bayesian Optimisation. The function f is expensive to evaluate and only accessible via noisy observations. The domain $\mathcal{X} \subseteq \mathbb{R}^d$ represents a constrained space defined by hardware inventory limits. Our objective is to find an optimal configuration of GPU allocations that maximizes the function f .

Intro to Bayesian Optimisation

SAASBO

4.8.1 Direct Island Generation

In the direct island generation approach, islands are explicitly defined within the Bayesian Optimisation process. Configurations correspond to a fixed number of islands, with each candidate solution representing a specific allocation of resources. The number of GPUs in each island is directly part of the output space, where specific variables represent the GPU count per slot. This setup provides precise control over how resources are grouped, but it also requires well-defined constraints to ensure that all generated configurations are valid.

Parameters

- G : the set of GPU types, e.g., $G = \{\text{A100}, \text{V100}, \dots\}$
- K : the number of GPU slots per type
- $x_{g,k}$: the integer decision variable representing the number of GPUs of type $g \in G$ assigned to slot $k \in \{1, \dots, K\}$
- I_g : the total inventory of GPU type $g \in G$

Each parameter $x_{g,k}$ is bounded:

$$x_{g,k} \in \{0, 1, \dots, I_g\}, \quad \forall g \in G, \forall k \in \{1, \dots, K\}$$

Constraints

For each GPU type $g \in G$, we enforce that the total number of GPUs allocated across all K slots does not exceed the inventory I_g :

$$\sum_{k=1}^K x_{g,k} \leq I_g, \quad \forall g \in G$$

How the first version of this worked...

What are the problems with it...

4.8.2 Divider Island Generation

The divider island generation method uses a more abstract representation that substantially reduces the dimensionality of the problem. Explicit allocation at the slot level requires one parameter per slot per GPU type, resulting in a parameter space of size $|G| \times K$. This approach becomes computationally expensive due to the high number of variables and the need for constraints to ensure valid configurations.

To address this, the divider-based approach reduces the search space to just two parameters per GPU type: the total number of islands and a scalar that controls the balance of GPU allocation across those islands. This formulation results in a significantly smaller parameter space of size $|G| \times 2$, enabling more efficient optimisation and faster convergence.

An important advantage of this method is that it avoids the need for explicit resource constraints, which are otherwise necessary to enforce inventory limits and prevent infeasible configurations. By shifting from low-level allocation to higher-level distribution strategies, divider island generation maintains control over performance-critical characteristics while offering a more scalable and interpretable optimisation process.

Input Variables

Let G_{total} denote the total number of GPUs available for that type, and let N_{target} represent the desired number of islands to create. We can allow the actual number of islands may be lower if there are not enough GPUs to satisfy the minimum-size requirement. The parameter S_{skew} controls how any leftover GPUs (after each island has been assigned its minimum) are distributed: when $S_{\text{skew}} \approx 0.0$, the extra GPUs are spread as evenly as possible; if $S_{\text{skew}} > 0.0$, islands with higher indices receive proportionally more of the remainder; and if $S_{\text{skew}} < 0.0$, islands with lower indices receive more. Finally, I_{min} specifies the minimum number of GPUs that any island must contain.

Preconditions

Initially, we verify that the input parameters meet a series of constraints, if any of these conditions is not met, the procedure raises an error and terminates immediately. The requirement $I_{\text{min}} > 0$ ensures that each island contains at least one GPU, since a non-positive minimum would be impossible. Similarly, G_{total} must be nonnegative because a negative total GPU count is invalid, and N_{target} must also be nonnegative because a negative number of islands cannot be requested.

Next, we perform trivial checks to assess whether it is possible to create at least one island given the input parameters. If $N_{\text{target}} = 0$, then no islands are to be created and the function returns the empty set $\emptyset$. The procedure then addresses cases in which allocation is not possible: if $G_{\text{total}} = 0$, there are no GPUs to allocate and the function

returns $\emptyset$. Similarly, if $G_{\text{total}} < I_{\text{min}}$, there are insufficient GPUs to form even a single island of size I_{min} , so the function again returns $\emptyset$. Only when all these preliminary checks have passed does the algorithm proceed to determine the feasible number of islands and distribute the GPUs accordingly.

Number of Islands

To establish how many islands can feasibly be created given the available GPUs and the minimum-size requirement, we first compute $N_{\text{max_possible}} = \lfloor G_{\text{total}} / I_{\text{min}} \rfloor$, which represents the maximum number of islands if each island is allocated exactly I_{min} GPUs. We then set $N_{\text{actual}} = \min(N_{\text{target}}, N_{\text{max_possible}})$, so that the actual number of islands cannot exceed either the user's target or the physical limit imposed by the GPU pool. If $N_{\text{actual}} = 0$, this indicates that even a single island of size I_{min} cannot be formed (either because $N_{\text{target}} = 0$ or because $G_{\text{total}} < I_{\text{min}}$), and the function returns the empty set $\emptyset$. Otherwise, N_{actual} islands will be created, each initially allocated I_{min} GPUs before any further distribution of the remaining GPUs occurs.

Base Allocation

In the base allocation step, we allocate I_{min} GPUs to each of the N_{actual} islands, so that the total GPUs used for this initial distribution is $G_{\text{base_used}} = N_{\text{actual}} \cdot I_{\text{min}}$. The number of GPUs remaining to be distributed is then $G_{\text{remaining}} = G_{\text{total}} - G_{\text{base_used}}$. At this point, every island has been provisioned with its minimum allocation of I_{min} GPUs, and the remaining $G_{\text{remaining}}$ GPUs will be allocated in subsequent steps.

Island Distribution

In the case where $G_{\text{remaining}} = 0$, all GPUs have already been assigned to the N_{actual} islands at their minimum size, so we make no weighted adjustments. When $N_{\text{actual}} = 1$, there is only a single island so we add all $G_{\text{remaining}}$ GPUs to that island. Otherwise we can distribute the remaining GPUs using the S_{skew} value.

The distribution begins with the calculation of a weight w_k for each island k , where k ranges from 1 to N_{actual} , computed as $w_k = k^{S_{\text{skew}}}$. However, if S_{skew} is numerically close to zero, each island is given an equal weight of $w_k = 1.0$. These weights are then normalised so that we can derive proportions p_k , indicating each island's share of the remaining GPUs. The sum of the weights is computed as

$$W_{\text{sum}} = \sum_{j=1}^{N_{\text{actual}}} w_j.$$

Each proportion is determined by dividing the weight of an island by the weight sum:

$$p_k = \frac{w_k}{W_{\text{sum}}}.$$

Using these proportions, we compute the ideal (possibly fractional) allocation s_k^{ideal} for each island:

$$s_k^{\text{ideal}} = p_k \cdot G_{\text{remaining}}.$$

Integer Adjustment

Because GPUs can only be allocated in integer units, we use the largest remainder method to convert the fractional allocations into integers. First, each island k is provisionally assigned a base number of GPUs:

$$s_k^{\text{base.int}} = \lfloor s_k^{\text{ideal}} \rfloor.$$

The current island sizes (initialised to $I_{\min}$) are then increased by these integer allocations.

Parameters

Let:

- G : the set of GPU types, e.g., $G = \{\text{A100}, \text{V100}, \dots\}$
- n_g : the number of islands for GPU type $g \in G$
- s_g : a scalar value representing the skew or balance in GPU allocation across the n_g islands for GPU type g
- I_g : the total available inventory of GPU type g

The optimization parameters for each GPU type $g \in G$ are:

$$(n_g, s_g) \in \mathbb{Z}_{\geq 1} \times [0, 1]$$

These two parameters per GPU type define the entire resource distribution strategy, allowing the Bayesian Optimization process to search over high-level allocation patterns while implicitly satisfying inventory constraints by design.

Chapter 5

Evaluation

To do...

Repeat every benchmark 10 times as they are non-deterministic

Compare heterogenous vs homogenous cases (show why heterogeneous is better)

Display any benchmarks specific to MoE

5.1 Inner Loop

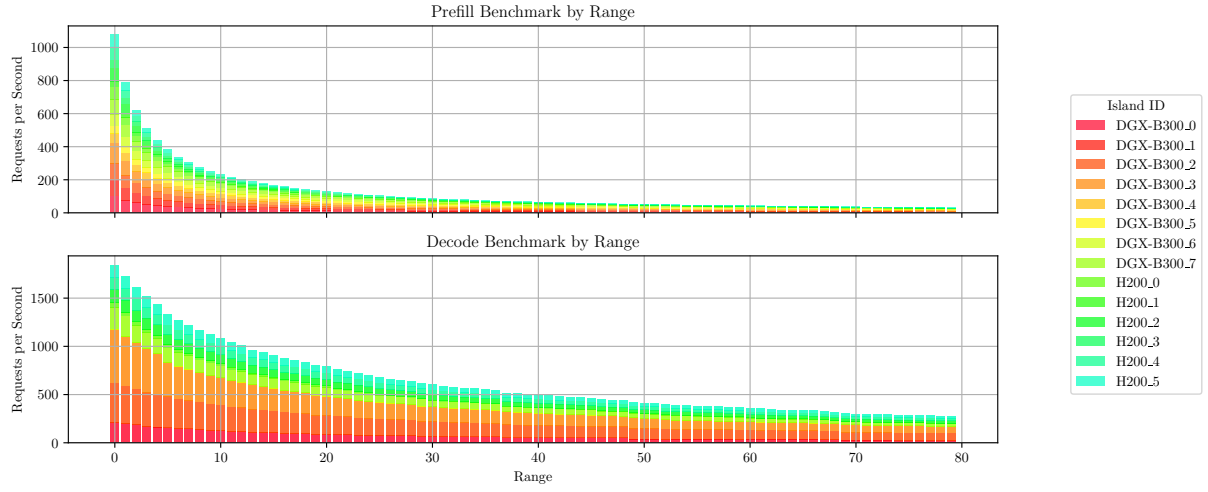


Figure 5.1: Sample Benchmark

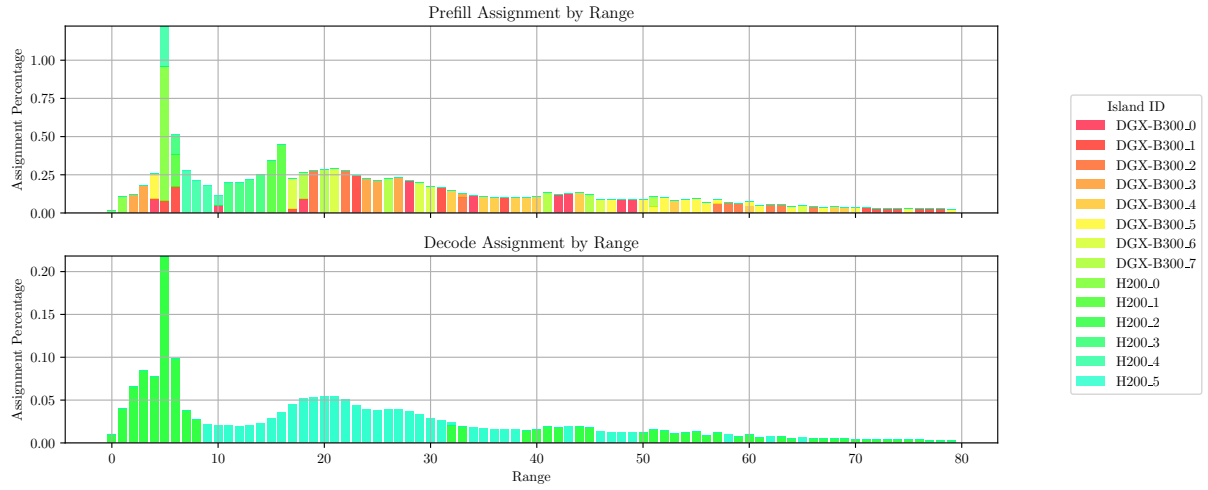


Figure 5.2: Sample Assignment

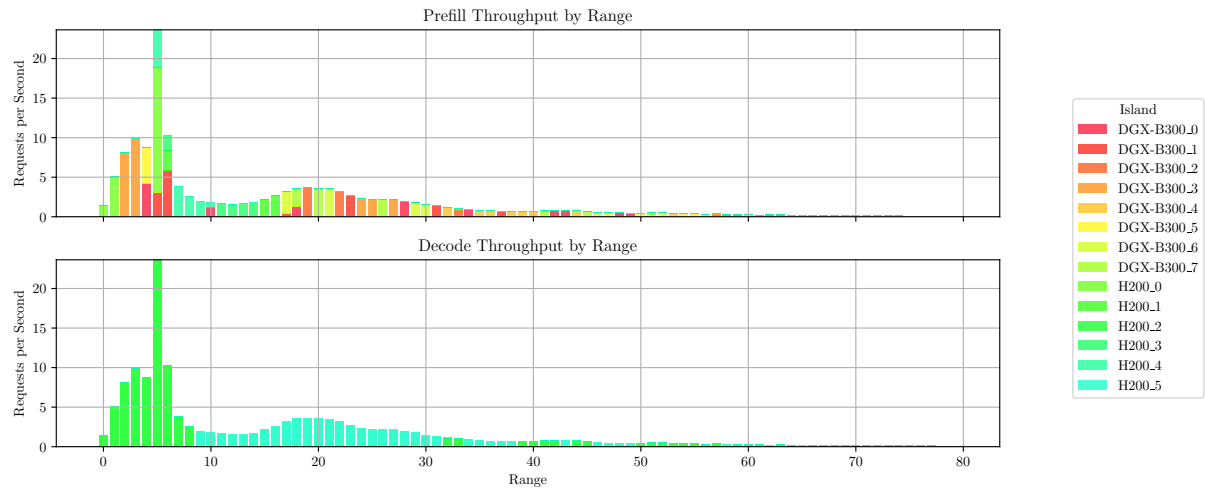


Figure 5.3: Sample Throughput

5.2 Outer Loop

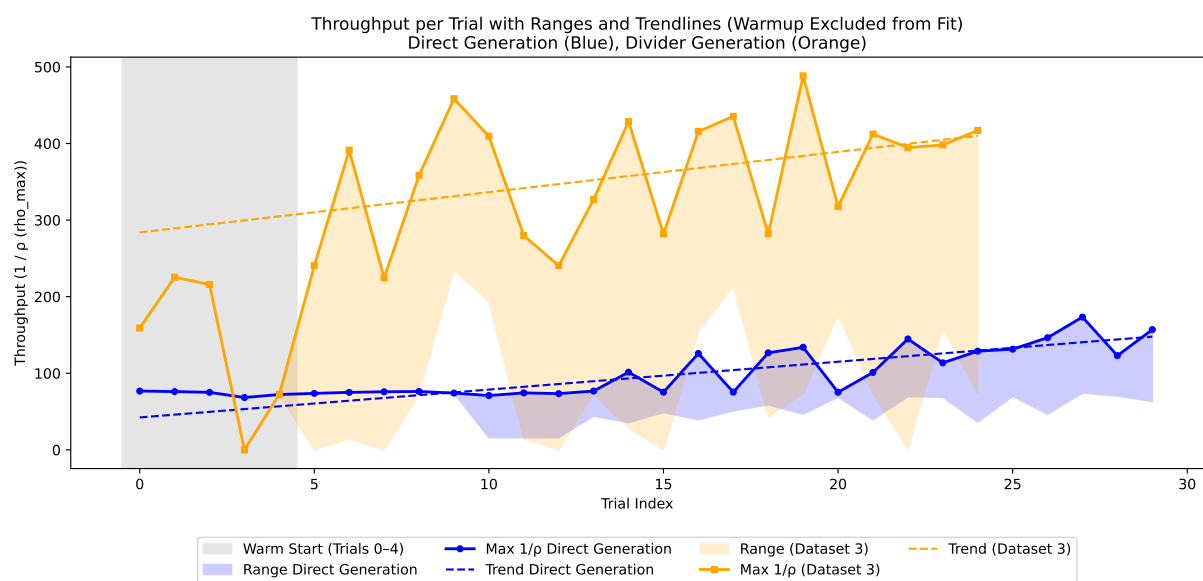


Figure 5.4: Comparison of BO methods

5.2.1 Traces

Azure LLM

Source [8]

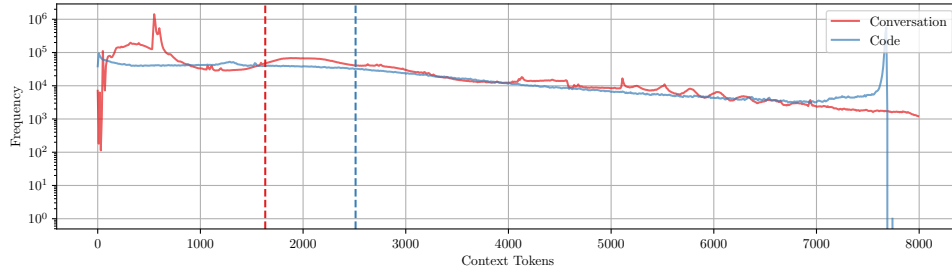


Figure 5.5: Context Tokens

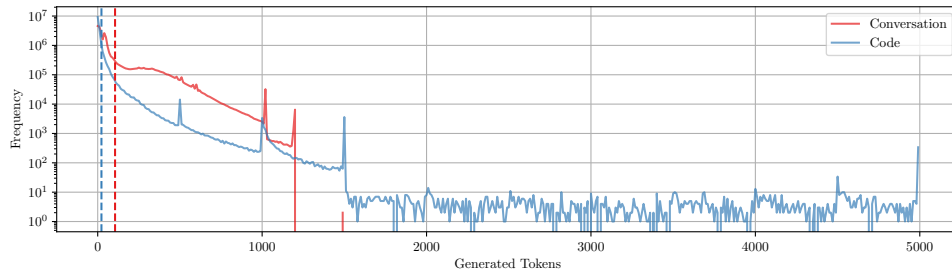
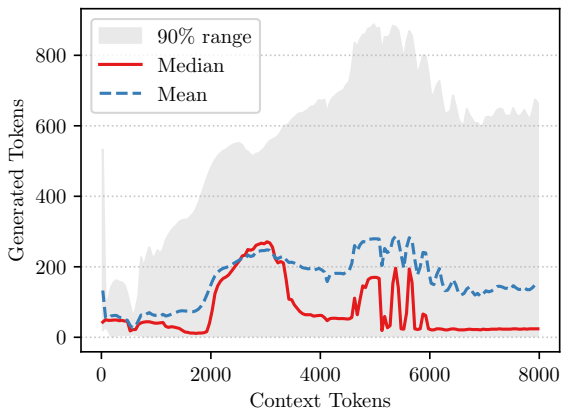
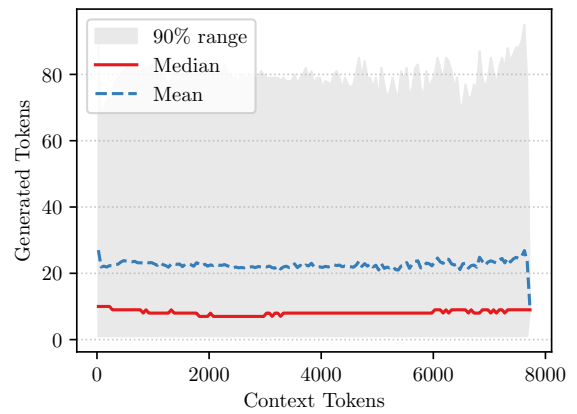


Figure 5.6: Decode Tokens



(a) Conversation Trace



(b) Code Trace

Figure 5.7: Comparison of Conversation and Code

BurstGPT

Source [9]

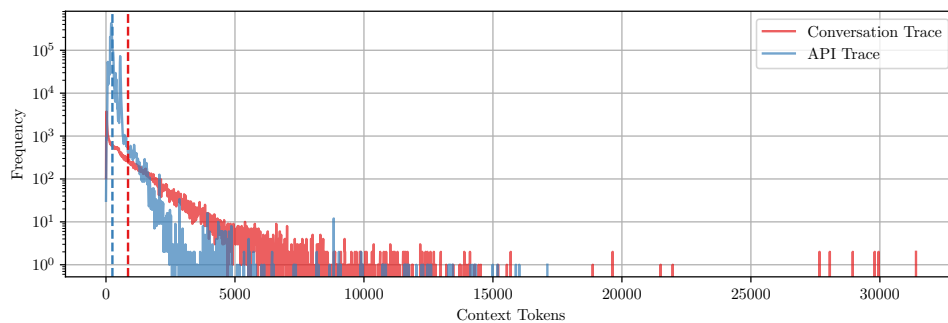


Figure 5.8: Context Tokens

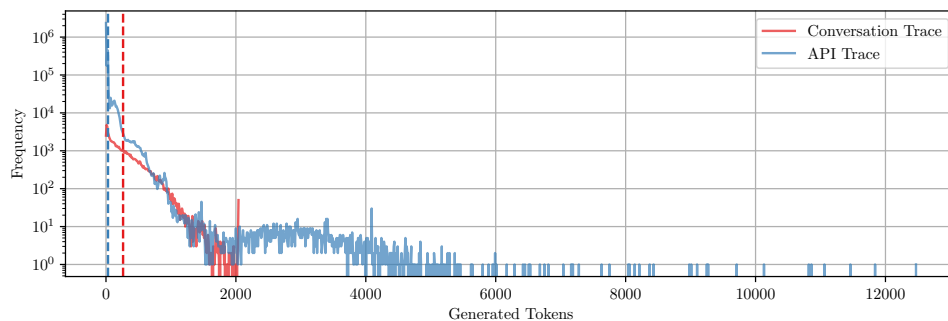
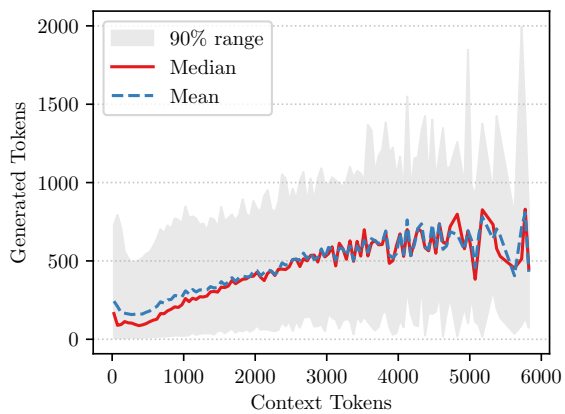
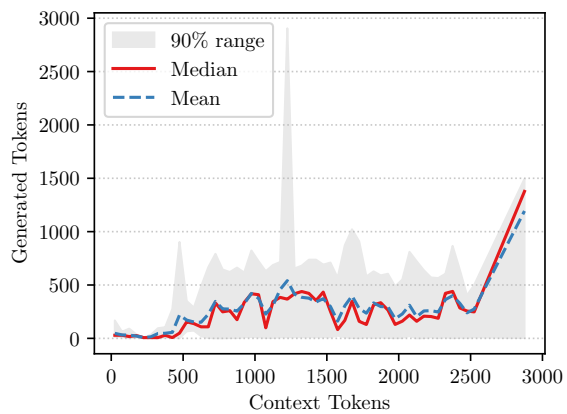


Figure 5.9: Decode Tokens



(a) Conversation Trace



(b) Code Trace

Figure 5.10: Comparison of Conversation and Code

Chapter 6

Summary and conclusions

References

- [1] Y. Jiang *et al.* “ThunderServe: High-performance and Cost-efficient LLM Serving in Cloud Environments.” Comment: MLSys 2025. arXiv: 2502.09334 [cs]. (Feb. 13, 2025), <http://arxiv.org/abs/2502.09334> (visited on May 13, 2025), pre-published.
- [2] Y. Jiang *et al.* “HexGen: Generative Inference of Large Language Model over Heterogeneous Environment.” Comment: Accepted by ICML 2024. arXiv: 2311.11514 [cs]. (May 27, 2024), <http://arxiv.org/abs/2311.11514> (visited on Nov. 18, 2024), pre-published.
- [3] Y. Zhong *et al.* “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving.” Comment: OSDI 2024. arXiv: 2401.09670 [cs]. (Jun. 6, 2024), <http://arxiv.org/abs/2401.09670> (visited on Jun. 3, 2025), pre-published.
- [4] P. Patel *et al.* “Splitwise: Efficient generative LLM inference using phase splitting.” Comment: 12 pages, 19 figures. arXiv: 2311.18677 [cs]. (May 20, 2024), <http://arxiv.org/abs/2311.18677> (visited on Jun. 3, 2025), pre-published.
- [5] A. Agrawal *et al.* “Vidur: A Large-Scale Simulation Framework For LLM Inference.” arXiv: 2405.05465 [cs]. (May 21, 2024), <http://arxiv.org/abs/2405.05465> (visited on Nov. 19, 2024), pre-published.
- [6] icezack12, *Icezack12/shallowsim*, Jun. 3, 2025. <https://github.com/icezack12/shallowsim> (visited on Jun. 3, 2025).
- [7] H. Zhang *et al.* “A Hardware Evaluation Framework for Large Language Model Inference.” arXiv: 2312.03134 [cs]. (Dec. 5, 2023), <http://arxiv.org/abs/2312.03134> (visited on Jun. 3, 2025), pre-published.
- [8] J. Stojkovic *et al.* “DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency.” arXiv: 2408.00741 [cs]. (Aug. 1, 2024), <http://arxiv.org/abs/2408.00741> (visited on Jun. 3, 2025), pre-published.
- [9] Y. Wang *et al.* “BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems.” arXiv: 2401.17644 [cs]. (May 26, 2025), <http://arxiv.org/abs/2401.17644> (visited on Jun. 3, 2025), pre-published.

Appendix A

Technical details, proofs, etc.

Appendices are for optional materials that is not essential to understanding the work, and that the examiners are not expected to read, but that will be of value to readers interested in additional, in-depth technical detail.

A.1 Lorem ipsum

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.